NLP – Assignment: Identifying Key Entities in Recipe Data

Submitted by: Kandarp Joshi

Date: 04/11/2025

<u>Business Objective</u>:

The goal of this assignment is to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities from recipe data. The model will classify words into predefined categories such as ingredients, quantities and units, enabling the creation of a structured database of recipes and ingredients that can be used to power advanced features in recipe management systems, dietary tracking apps, or e-commerce platforms.

DataFrame Description after loading the data:

```python
# print the information of the dataframe
ing_df.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 285 entries, 0 to 284
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   input   285 non-null    object
 1   pos     285 non-null    object
dtypes: object(2)
memory usage: 4.6+ KB
```

Adding Input Tokens and pos tokens:

2.2.1 Create input_tokens and pos_tokens columns by splitting the input and pos from the dataframe [3 marks]

Split the input and pos into input_tokens and pos_tokens in the dataframe and display it in the dataframe

```python
# split the input and pos into input_tokens and pos_tokens in the dataframe
nlp = spacy.load("en_core_web_sm")
# Tokenize input
ing_df['input_tokens'] = ing_df['input'].apply(lambda x: [token.text for token in nlp(x)])

# Tokenize POS
ing_df['pos_tokens'] = ing_df['pos'].apply(lambda x: [token.text for token in nlp(x)])

ing_df.head()
```

Edit ⌘I Chat ⌘L

✓ 3.9s Python

	input	pos	input_tokens	pos_tokens
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient ingredient ingredient

Data Observations: After Initial data analysis, we observe that the Data in the JSON file is unstructured, containing complex combination of ingredients, quantity, units and detailed texts in a single input column. As the data is in array of objects, the indexes are 0,1,2. All the 3 records require cleaning, formatting and decomposition.

Training Dataset validation and overview:

```
 Training Dataset Validation :
Dataset shape: (142, 6)

Length Validation:
Number of input_token sequences: 142
Number of pos_token sequences: 142
Total flattened input_tokens: 4632
Total flattened pos_tokens: 4632
 Sequence counts match between input_tokens and pos_tokens
Flattened token counts match

First 10 input_tokens:
  1: ['1', 'cup', 'Gram', 'flour', 'besan', '1/2', 'teaspoon', 'Ajwain', 'Carom', 'seeds', '1/4', 'Red', 'Chilli', 'powder', '2', 'tablespoons', '
  2: ['1', 'eggplant', '2', 'inch', 'ginger', 'onion', 'tomato', 'green', 'chillies', 'teaspoon', 'turmeric', 'powder', 'garam', 'masala', 'coriar
  3: ['2', 'onions', '1', 'teaspoon', 'cumin', 'seeds', 'tablespoons', 'black', 'urad', 'dal', 'split', 'tamarind', 'paste', '3', 'dry', 'red', '
  4: ['2', 'cups', 'Makki', 'Ka', 'Atta', 'Yellow', 'Corn', 'Meal', 'Flour', '1', '1/2', 'Methi', 'Leaves', 'Fenugreek', 'chopped', 'Green', 'Chil
  5: ['10', 'prawns', '1', 'tablespoon', 'soy', 'sauce', 'teaspoon', 'sesame', 'gingelly', 'oil', 'cloves', 'garlic', 'finely', 'chopped', '2', '
  6: ['2', 'cups', 'Whole', 'Wheat', 'Flour', 'Salt', 'Ghee', '1', 'tablespoon', '200', 'grams', 'Paneer', 'Homemade', 'Cottage', 'Cheese', '1/4',
  7: ['1/2', 'cup', 'Coriander', 'Dhania', 'Leaves', 'chopped', '4', 'Green', 'Chillies', '1', 'inch', 'Ginger', 'Salt', 'Lemon', 'juice', 'extra
  8: ['300', 'grams', 'Tofu', 'crumbled', '1', 'Onion', 'chopped', 'Red', 'Bell', 'pepper', 'Capsicum', 'Tomato', 'Green', 'Chilli', 'teaspoon', '
  9: ['500', 'grams', 'Baby', 'Potatoes', '2', 'Dry', 'Red', 'Chillies', 'teaspoons', 'Sesame', 'seeds', '1', 'teaspoon', 'Cumin', 'Fennel', '1/4'
  10: ['1', 'cup', 'rice', '2', 'tablespoons', 'ghee', 'teaspoon', 'cumin', 'seeds', 'bay', 'leaf', 'lentil', 'sugar', '3', 'long', 'green', 'chi
...
Training - Flattened input_tokens: 4632
Training - Flattened pos_tokens: 4632
Training - Sample input: ['1', 'cup', 'Gram', 'flour', 'besan', '1/2', 'teaspoon', 'Ajwain', 'Carom', 'seeds']
Training - Sample pos: ['quantity', 'unit', 'ingredient', 'ingredient', 'ingredient', 'quantity', 'unit', 'ingredient', 'ingredient', 'ingredient'
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Categorizing the tokens:

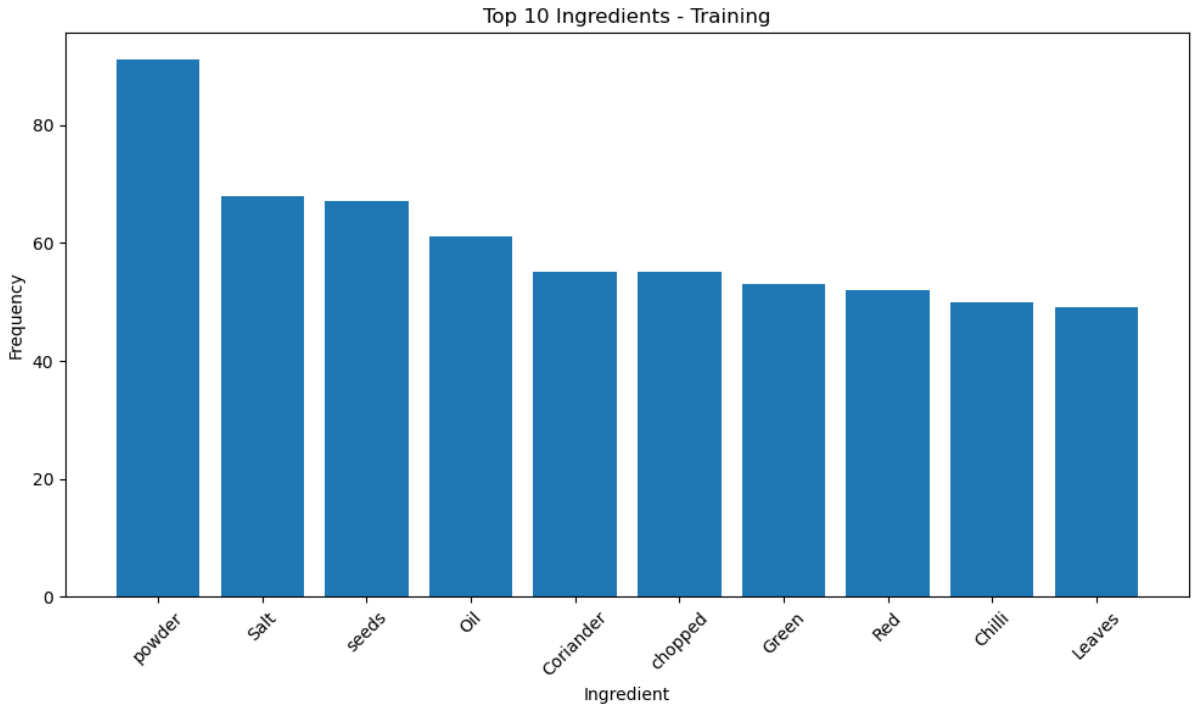
```
# call the function to categorise the labels into respective list

ingredients, units, quantities = categorize_tokens(flat_train_input_tokens, flat_train_pos_tokens)
print(f"Ingredients: {ingredients}")
print(f"Units: {units}")
print(f"Quantities: {quantities}")
    ⌘I for Command, ⌘L for Cascade
00]  ✓ 0.0s                                                                    Python

Ingredients: ['Gram', 'flour', 'besan', 'Ajwain', 'Carom', 'seeds', 'Red', 'Chilli', 'powder', 'Curd', 'Dahi', 'Yogurt', 'Sunflower', 'Oil', 'Gree
Units: ['cup', 'teaspoon', 'tablespoons', 'sprig', 'inch', 'teaspoon', 'tablespoon', 'sprigs', 'teaspoon', 'tablespoons', 'teaspoons', 'sprig', '
Quantities: ['1', '1/2', '1/4', '2', '4', '1', '2', '2', '1', '3', '1/2', '2', '1', '1/2', '10', '1', '2', '1/2', 'few', '2', '1', '200', '1/4', '
```

Top 5 Most Frequent items:

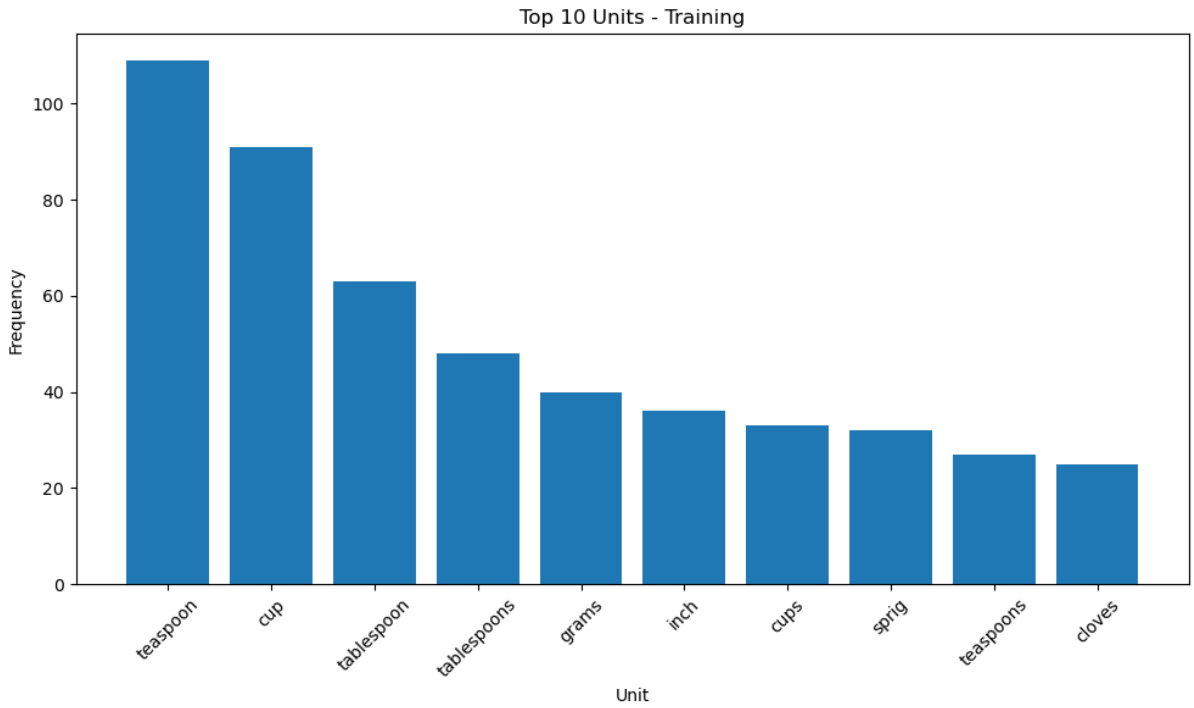
```
=== Training Dataset - Top 5 Ingredients ===
Total ingredients: 9
Unique ingredients: 5

Top 5 most frequent ingredients:
------------------------------
 1. salt                  :    3 (33.33%)
 2. oil                   :    3 (33.33%)
 3. onion                 :    1 (11.11%)
 4. garlic                :    1 (11.11%)
 5. tomato                :    1 (11.11%)

=== Training Dataset - Top 3 Units ===
Total units: 6
Unique units: 3

Top 3 most frequent units:
------------------------------
 1. cup                   :    3 (50.00%)
 2. tablespoon            :    2 (33.33%)
 3. teaspoon              :    1 (16.67%)

=== Training Dataset - Top 5 Quantitys ===
Total quantitys: 8
Unique quantitys: 4

Top 4 most frequent quantitys:
------------------------------
 1. 1                     :    3 (37.50%)
 2. 2                     :    3 (37.50%)
```

EDA Analysis:

Top 10 Ingredients:



Top 10 Units:



Creating Feature sets for CRF model:

```
Creating feature sets for CRF model...

1. Processing Training Data:
----------------------------------------
  Processed 25/142 training recipes
  Processed 50/142 training recipes
  Processed 75/142 training recipes
  Processed 100/142 training recipes
  Processed 125/142 training recipes

2. Processing Validation Data:
----------------------------------------
  Processed 25/62 validation recipes
  Processed 50/62 validation recipes

================================================
FEATURE EXTRACTION SUMMARY
================================================
Training recipes: 142
Validation recipes: 62
Sample training recipe length: 41 words
Features per word: 49 features
```

Computing Weight for labels:

```
Creating weight dictionary...
Total samples: 4632
Ingredient penalty factor: 2.0

Calculating weights:
----------------------------------------
quantity    : count=658    -> weight=7.040
unit        : count=563    -> weight=8.227
ingredient  : count=3411   -> inv_freq=1.358 -> penalized=2.716

{'ingredient': 2.7159190853122253}
```

Penalizing ingredient variable:

```
# penalise ingredient label
for label, weight in weight_dict.items():
    count = label_counts[label]
    penalty_info = " (PENALIZED)" if label.lower() == 'ingredient' else ""
    print(f"{label}: {weight:.3f}{penalty_info} (original count: {count})")

print(f"\nweight_dict = {weight_dict}")

✓ 0.0s

ingredient: 2.716 (PENALIZED) (original count: 3411)

weight_dict = {'ingredient': 2.7159190853122253}
```

Extract weighted features:

```
----------------------------------------
EXTRACTING WEIGHTED FEATURES FOR TRAINING DATA
----------------------------------------
Extracting features with class weights for Training...
Number of sequences: 142
Weight dictionary: {'ingredient': 2.7159190853122253}
  Processed 50/142 sequences...
  Processed 100/142 sequences...

Feature extraction with weights complete for Training:
  Processed sequences: 142
  Total words: 4632
  Weight statistics:
    Min weight: 1.000
    Max weight: 2.716
    Avg weight: 2.264

============================================================
EXTRACTING WEIGHTED FEATURES FOR VALIDATION DATA
============================================================
Extracting features with class weights for Validation...
Number of sequences: 62
Weight dictionary: {'ingredient': 2.7159190853122253}
  Processed 50/62 sequences...
...
  Weight statistics:
    Min weight: 1.000
    Max weight: 2.716
    Avg weight: 2.286
```

CRF Model Evaluation:

```
CRF MODEL EVALUATION - TRAINING DATA
================================================
Training Data Classification Report:
              precision    recall  f1-score   support

  ingredient     0.9924    1.0000    0.9962      3411
        unit     0.9946    0.9787    0.9866       563
    quantity     1.0000    0.9742    0.9869       658

    accuracy                         0.9937      4632
   macro avg     0.9957    0.9843    0.9899      4632
weighted avg     0.9938    0.9937    0.9937      4632
```

Confusion Matrix:

```
# create a confusion matrix on training datset

from sklearn_crfsuite import metrics

train_accuracy = metrics.flat_accuracy_score(y_train_labels, y_train_pred)
train_f1_weighted = metrics.flat_f1_score(y_train_labels, y_train_pred, average='weighted')
train_f1_macro = metrics.flat_f1_score(y_train_labels, y_train_pred, average='macro')

print(f"\nTraining Metrics Summary:")
print(f"Accuracy:        {train_accuracy:.4f}")
print(f"F1-Score (weighted): {train_f1_weighted:.4f}")
print(f"F1-Score (macro):    {train_f1_macro:.4f}")
```

Edit ⌘I Chat ⌘L

✓ 0.0s

```
Training Metrics Summary:
Accuracy:        0.9937
F1-Score (weighted): 0.9937
F1-Score (macro):    0.9899
```

CRF Model evaluation on validation dataset:

8.1 *Predict and Evaluate the CRF model on validation set* [3 marks]

Evaluate the metrics for CRF model by using flat classification report and confusion matrix

```python
# predict the crf model on validation dataset
y_val_pred = crf_weighted.predict(X_val_weighted_features)
```

✓ 0.0s

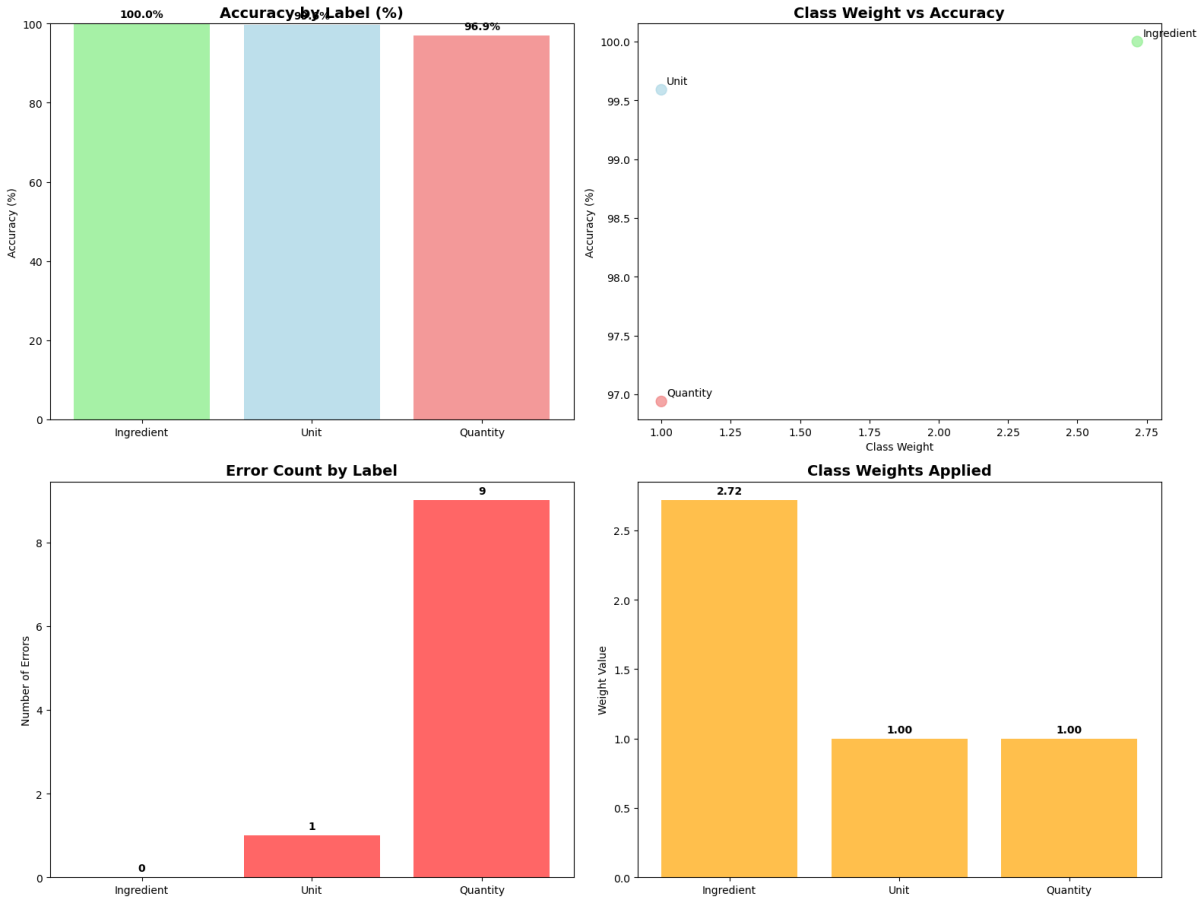
```python
# specify flat classification report
print("\nValidation Performance:")
print(flat_classification_report(y_val_labels, y_val_pred, digits=4))

# Store predictions for further analysis
validation_predictions = y_val_pred
```

✓ 0.0s

```
Validation Performance:
              precision    recall  f1-score   support

  ingredient     0.9951    1.0000    0.9975      1611
    quantity     1.0000    0.9694    0.9845       294
        unit     0.9918    0.9959    0.9939       244

    accuracy                         0.9953      2149
   macro avg     0.9956    0.9884    0.9919      2149
weighted avg     0.9954    0.9953    0.9953      2149
```

Error Analysis:

```
Converting error data to DataFrame...
Error DataFrame created:
  Shape: (10, 45)
  Columns: 45

 OVERALL VALIDATION ACCURACY ILLUSTRATION:
==================================================
Total Validation Tokens:    2,149
Correct Predictions:        2,139
Incorrect Predictions:      10

ACCURACY:                0.9953 (99.53%)
ERROR RATE:              0.0047 (0.47%)
```

```
 STARTING COMPREHENSIVE ERROR ANALYSIS
==================================================
ANALYZING VALIDATION ERRORS BY LABEL WITH CLASS WEIGHTS
================================================================
ERROR ANALYSIS BY LABEL:
==================================================

 INGREDIENT LABEL ANALYSIS:
   Class Weight:        2.716 (HIGH impact)
   Total Instances:     1,611
   Correct Predictions: 1,611
   Errors:              0
   Accuracy:            1.0000 (100.00%)
   Error Rate:          0.0000 (0.00%)
   Predicted Count:     1,619

 UNIT LABEL ANALYSIS:
   Class Weight:        1.000 (LOW impact)
   Total Instances:     244
   Correct Predictions: 243
   Errors:              1
   Accuracy:            0.9959 (99.59%)
   Error Rate:          0.0041 (0.41%)
   Predicted Count:     245
...

 DETAILED ERROR ANALYSIS WITH CONTEXT
==================================================
 No errors found - Perfect model performance!
```

Label performance visualizations: